TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

CÔNG NGHỆ MÃ NGUỒN MỞ

TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Chương 4

NGÔN NGỮ LẬP TRÌNH

python™

Mục tiêu

- Giới thiệu ngôn ngữ Python
- Ngôn ngữ Python dành cho ai
- Sử dụng được ngôn ngữ Python cơ bản

Nội dung

1. Giới thiệu ngôn ngữ Python
2. Cơ bản về ngôn ngữ Python
3. Kiểu dữ liệu
4. Toán tử và biểu thức
5. Cấu trúc điều khiển
6. Định nghĩa hàm
7. Xử lý file

Giới thiệu Python

- Python đã được Guido van Rossum tạo ra vào những năm 1980 tại Trung tâm CWI (Centrum Wiskunde & Informatica) ở Hà Lan.
- Được triển khai vào 12/1989. Lần đầu ra mắt vào năm 1991.
- Python là một ngôn ngữ lập trình bậc cao. Ngôn ngữ thông dịch.
- Ban đầu Python được phát triển chỉ chạy trên UNIX. Nhưng hiện nay Python chạy trên hầu hết các HĐH.
- Python 2.0 được ra mắt vào ngày 16/10/2000
- Python 3.0 được ra mắt vào ngày 3/12/2008
- Hiện nay, Python 3.12.0
- Python được quản lý bởi Python Software Foundation

Guido van Rossum

1 Giới thiệu Python

Hình ảnh từ website chính thức của Python https://www.python.org/about/

1 Giới thiệu Python

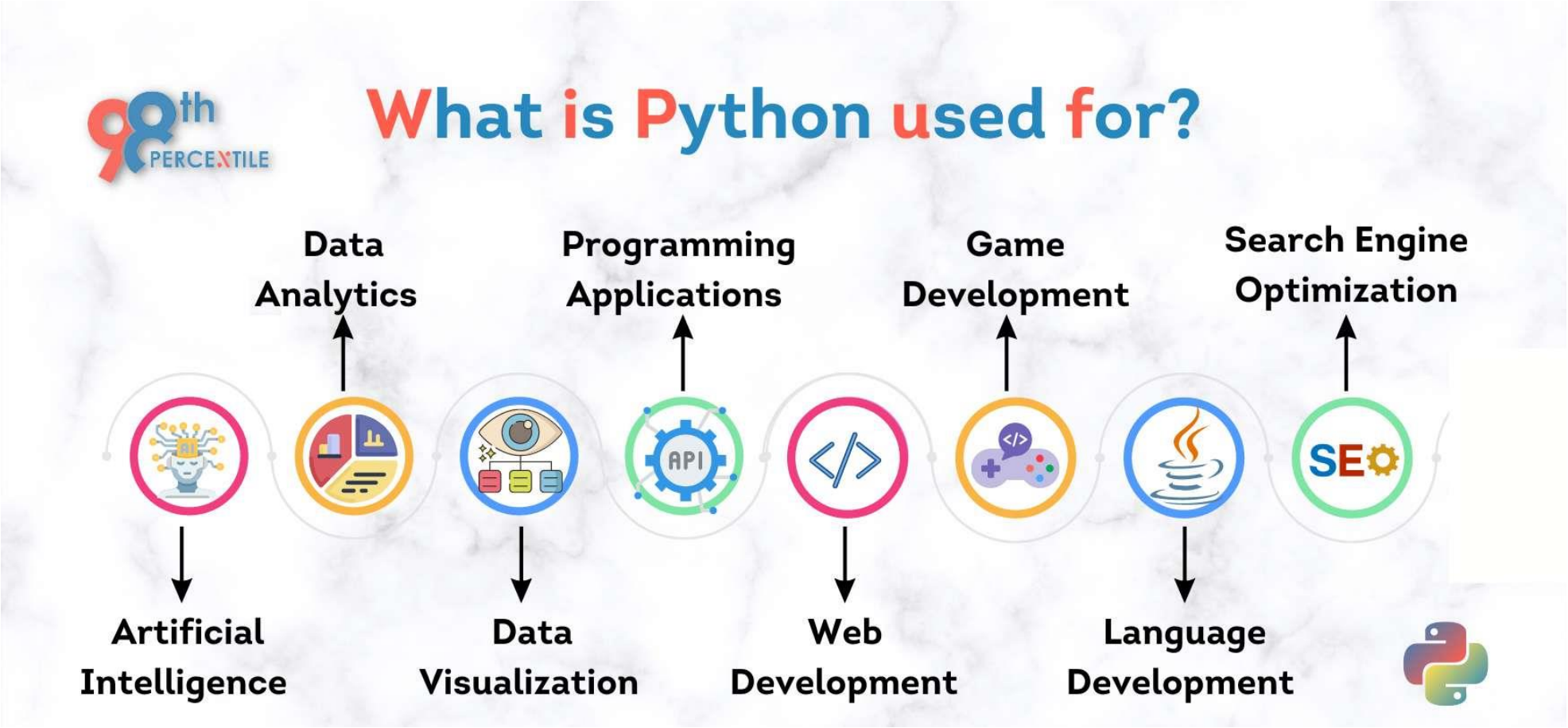
Đặc điểm của ngôn ngữ Python

Nguồn: Data Flair

- Python dễ đọc, dễ học và dễ nhớ.
- Hình thức rất sáng sủa, cấu trúc rõ ràng
- Phù hợp cho người mới học lập trình.
- Python còn cho phép người sử dụng viết mã lệnh với số lần gõ phím tối thiểu.

1 Giới thiệu Python

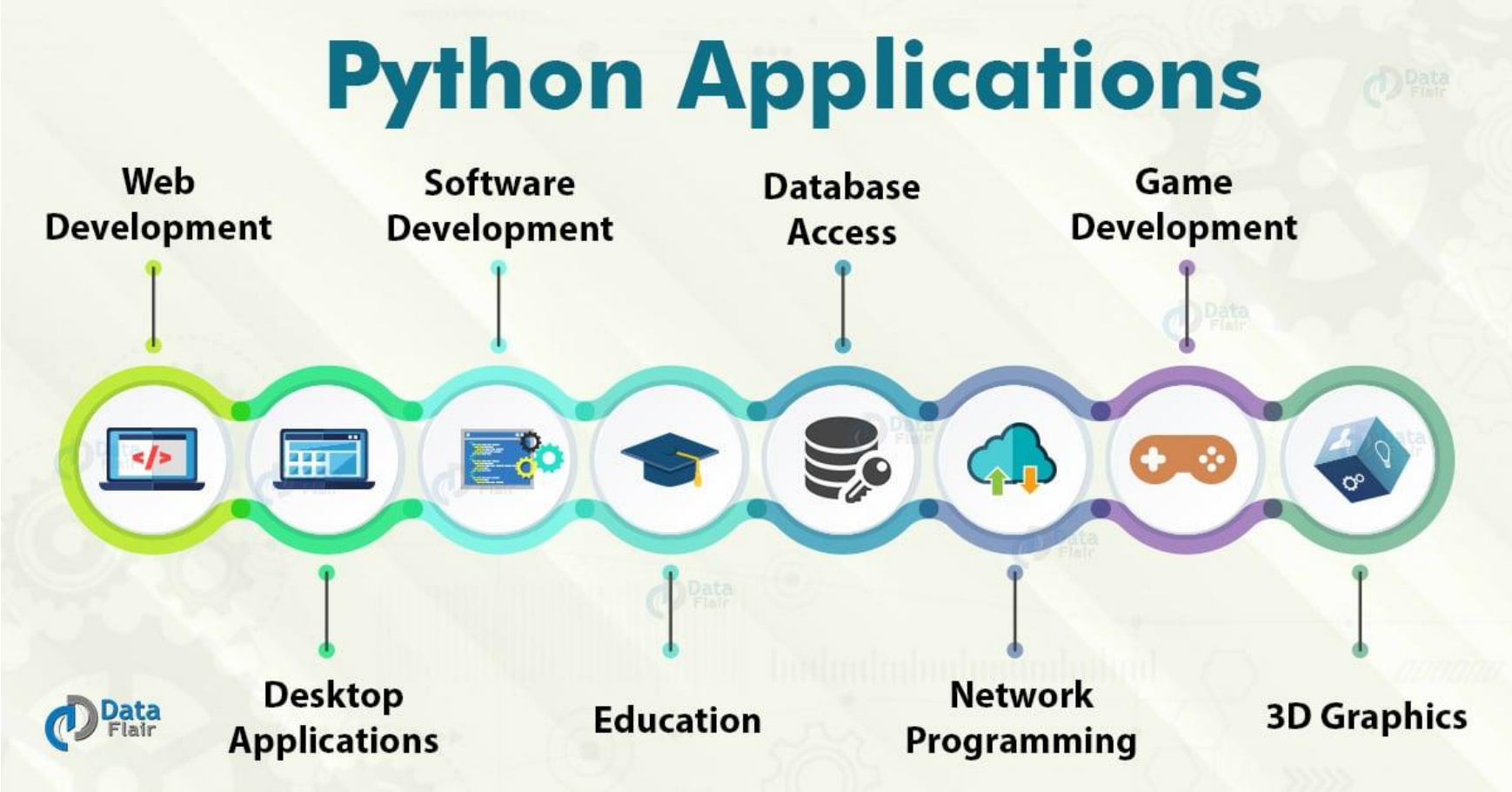
Python có thể làm được gì



https://www.98thpercentile.com/blog/what-is-python-used-for/

1 Giới thiệu Python

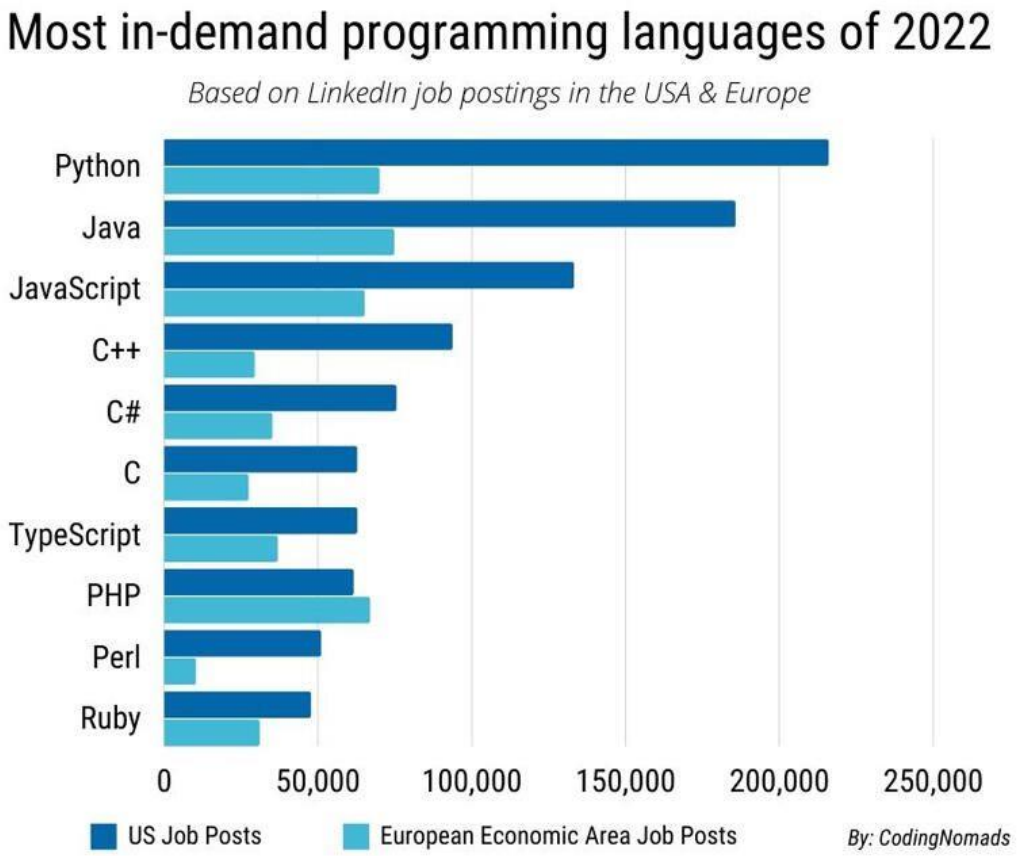
Python có thể làm được gì

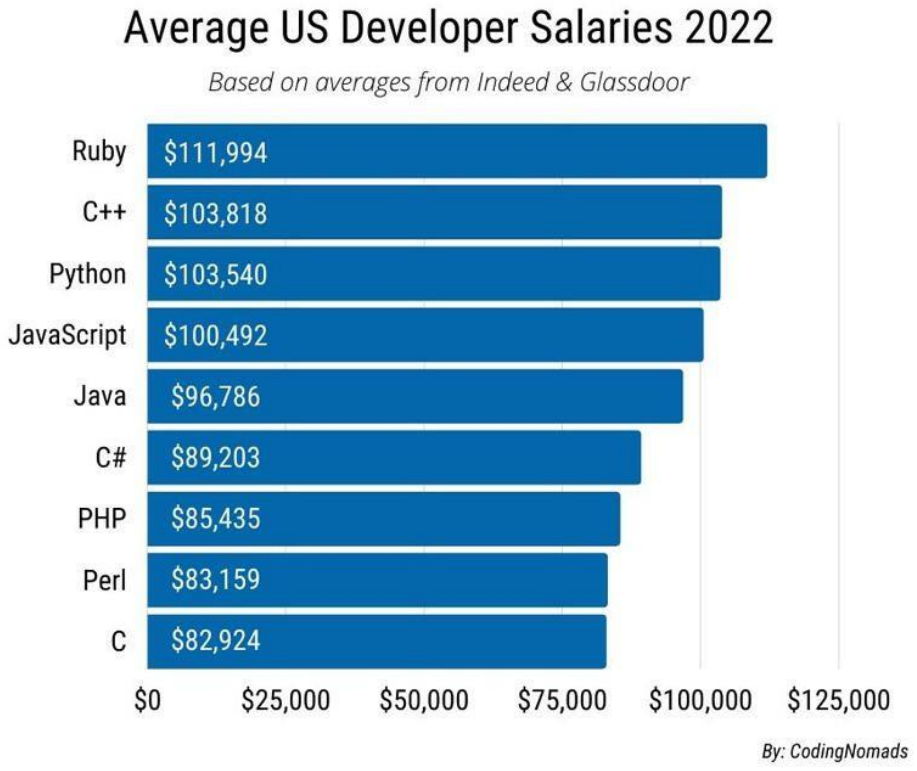


Nguồn: https://data-flair.training/blogs/python-applications/

1 Giới thiệu Python

Một vài thống kê liên quan đến ngôn ngữ Python

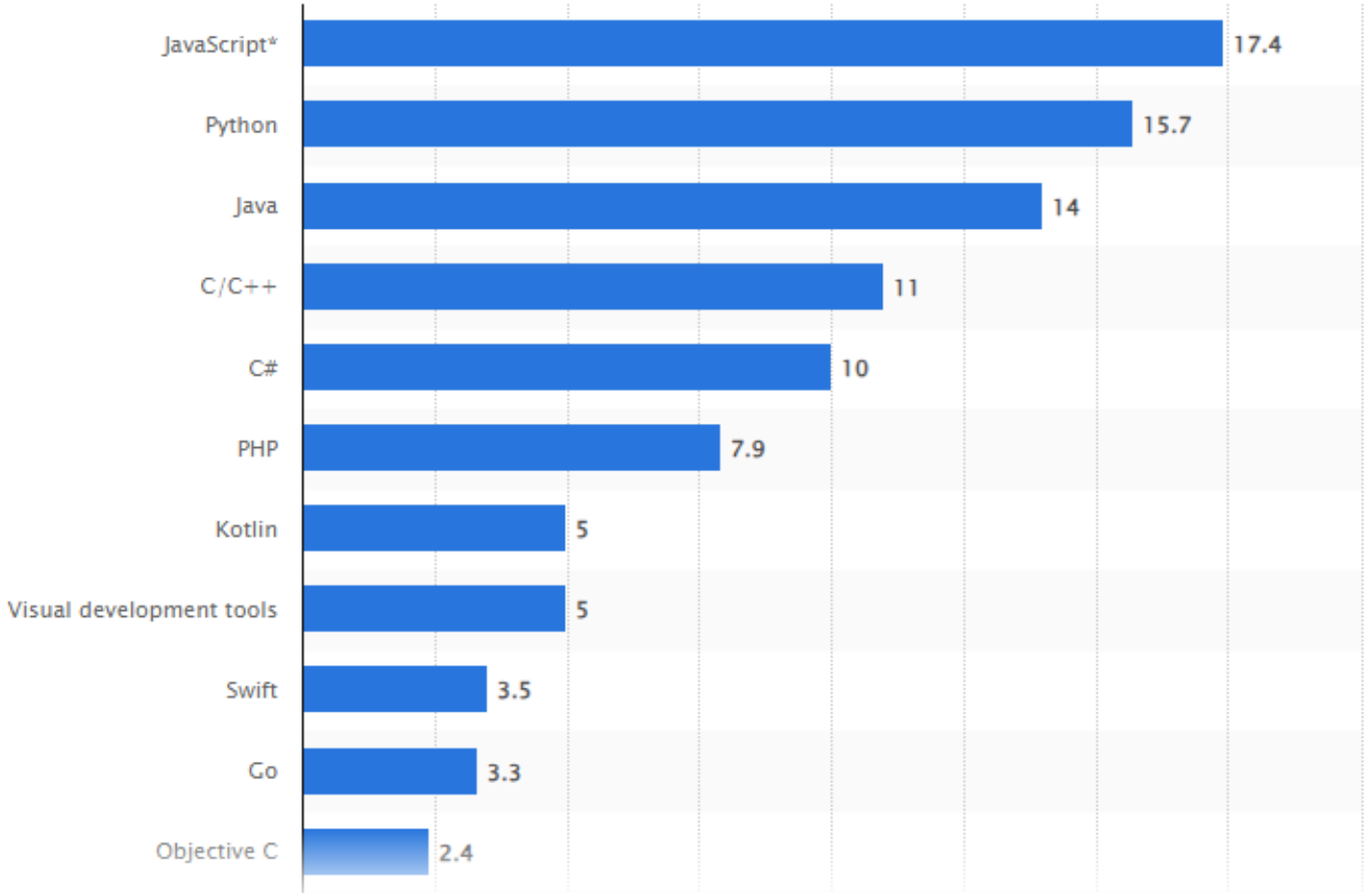




https://www.techrepublic.com/article/the-best-programming-languages-to-learn-in-2022/

1 Giới thiệu Python

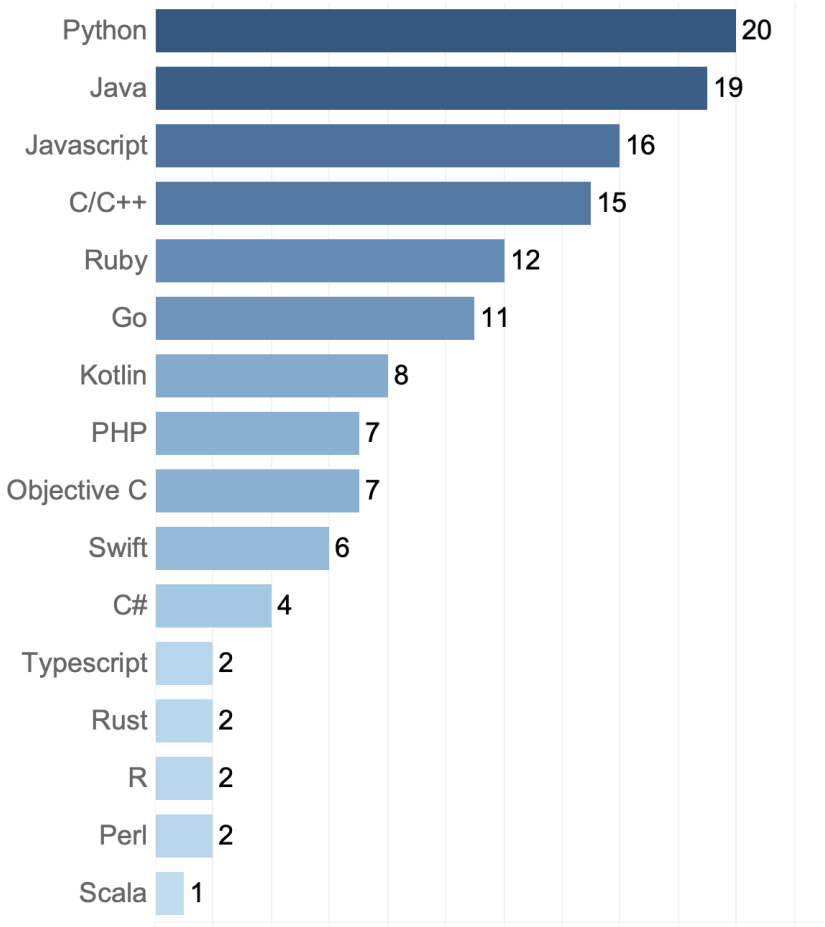
Size of programming language communities worldwide as of 2022*(in millions)*



https://www.statista.com/statistics/1241923/worldwide-software-developer-programming-language-communities/

1 Giới thiệu Python

Most Popular Programming Languages Used by Top 25 Unicorn Companies



https://www.codingdojo.com/blog/unicorn-languages-report

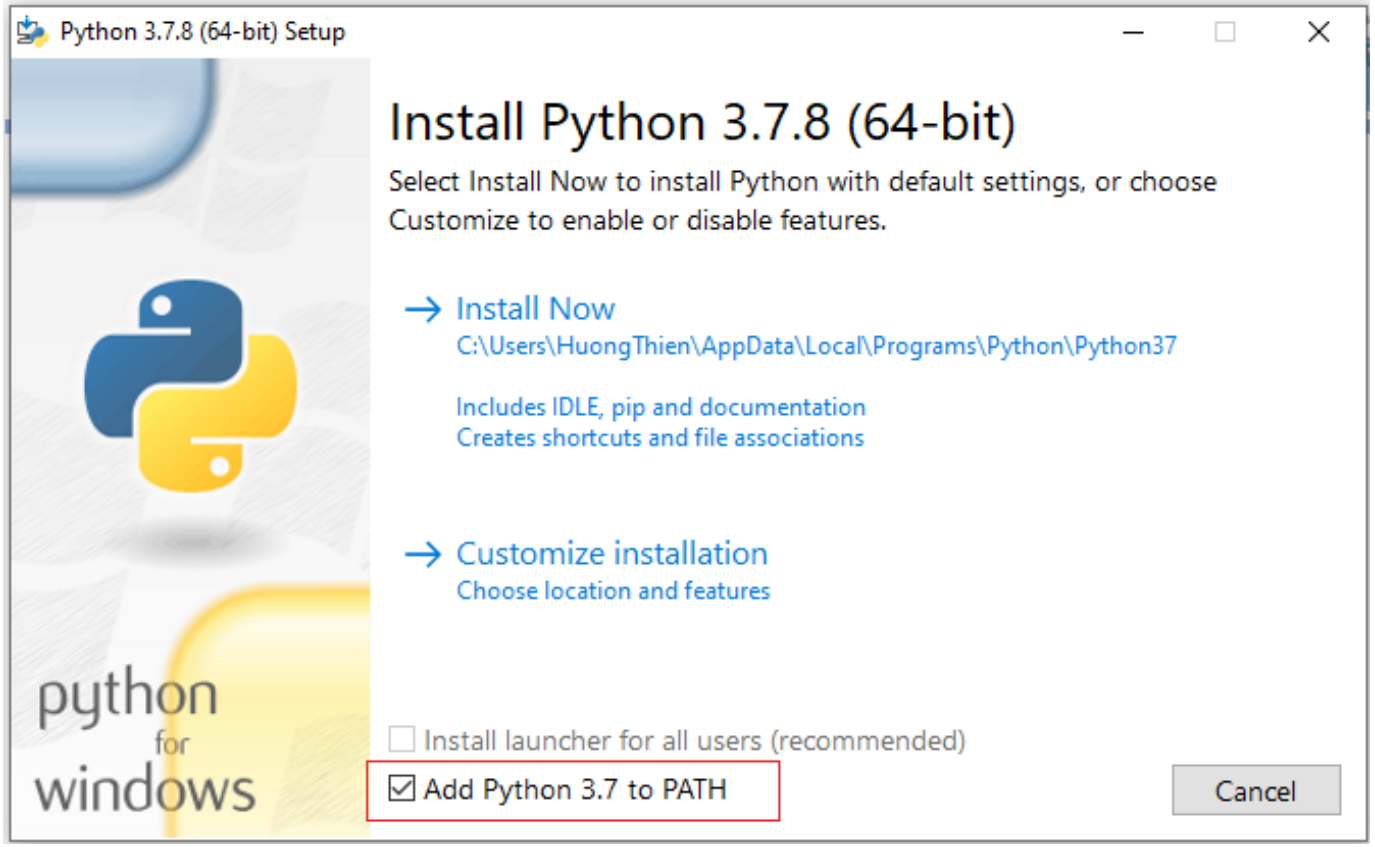
2 Cơ bản về ngôn ngữ Python

- Cài đặt Python
- Một vài phần mềm IDE
- Quản lý các package – Cài đặt package
- Biến: quy ước đặt tên, Không cần khai báo kiểu dữ liệu, xem kiểu dữ liệu của biến

Open Source Technologies

Cài đặt Python

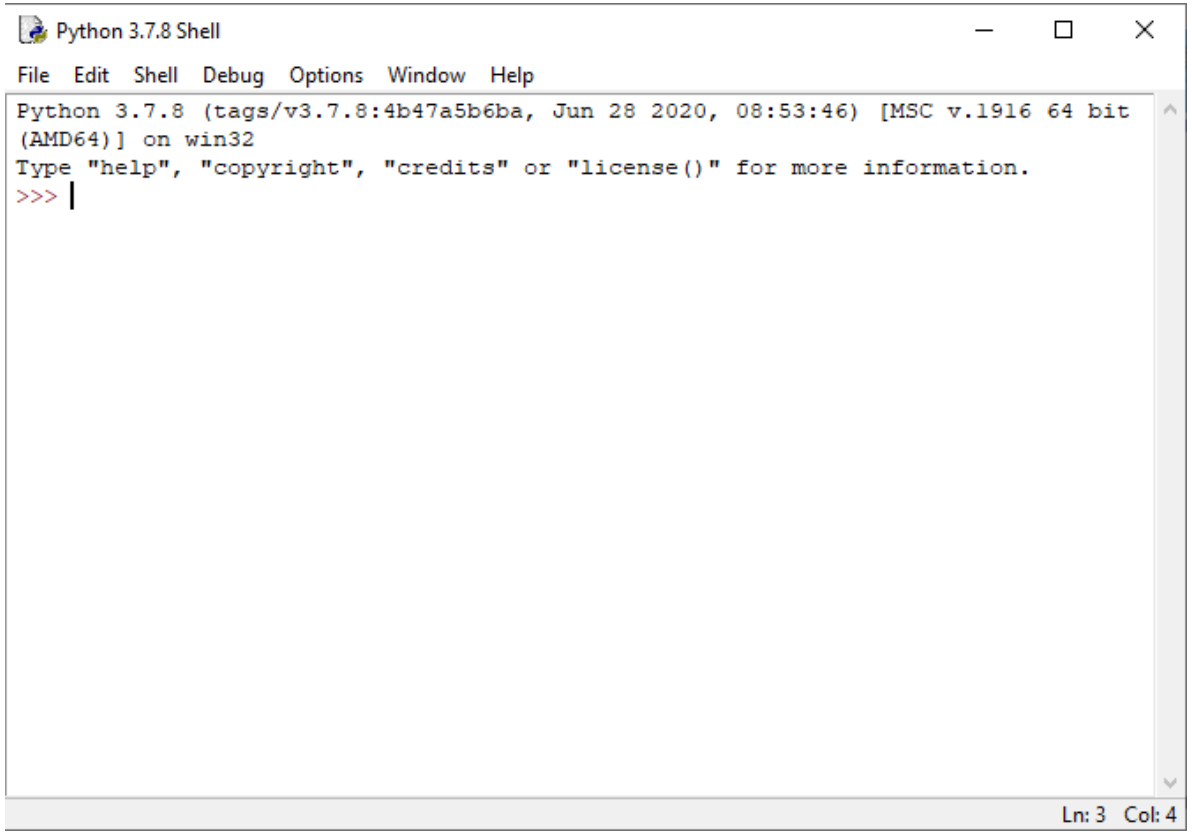
Download từ website
https://www.python.org/downloads/



Lưu ý: Tick chọn vào Add Python…to PATH

Open Source Technologies

Một vài IDE cho ngôn ngữ Python



IDLE

- IDLE là IDE mặc định của Python.
- IDLE sẽ được cài theo khi cài đặt Python.
- Hỗ trợ đa nền tảng: Windows, Linux, Mac
- Chỉ có những tính năng cơ bản.
- Hỗ trợ 2 loại file .py và .txt

Open Source Technologies

Một vài IDE cho ngôn ngữ Python

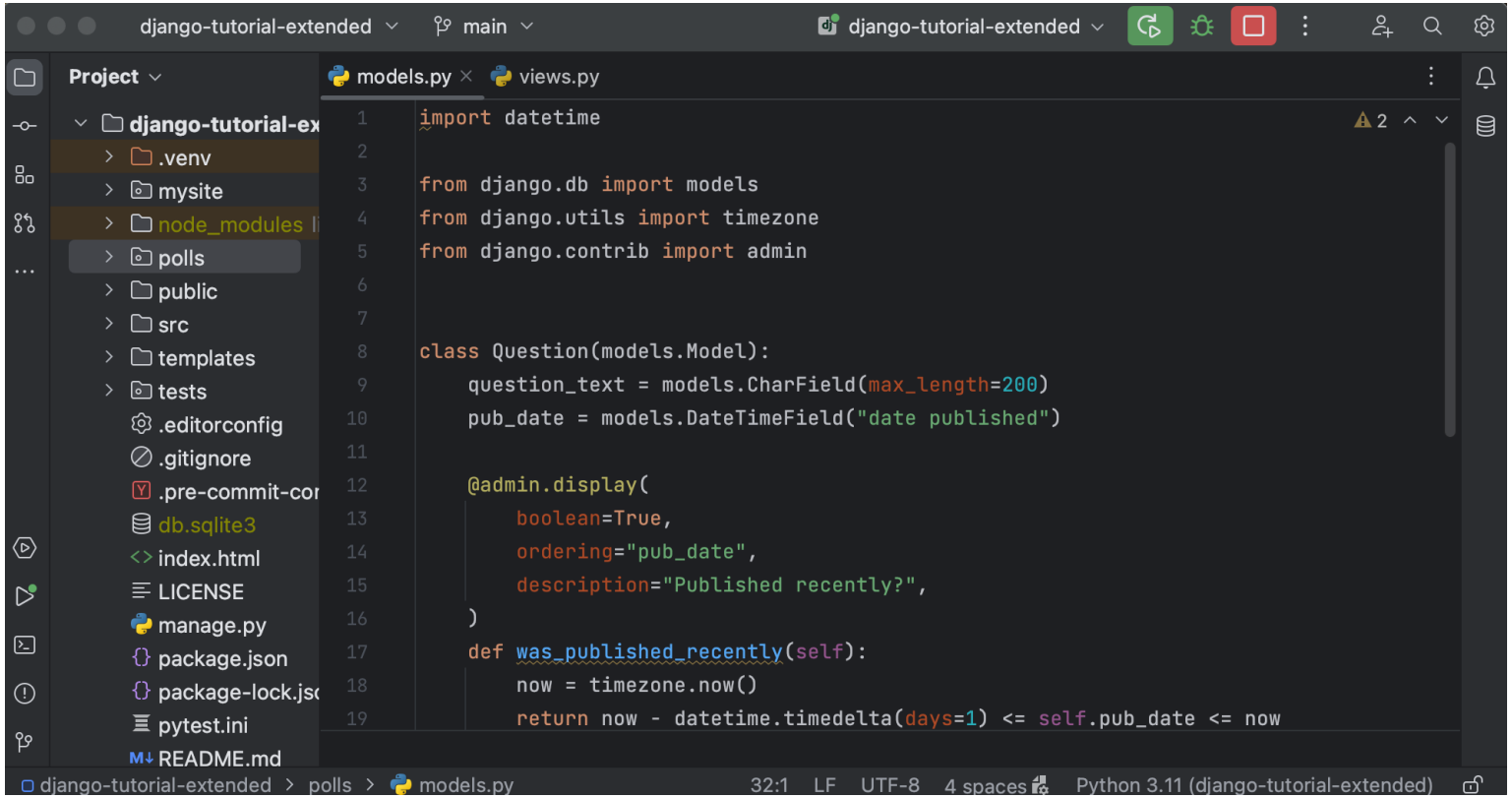
PyCharm

PyCharm Professional

The Python IDE for Professional Developers

PyCharm Community Edition

The IDE for Pure Python Development



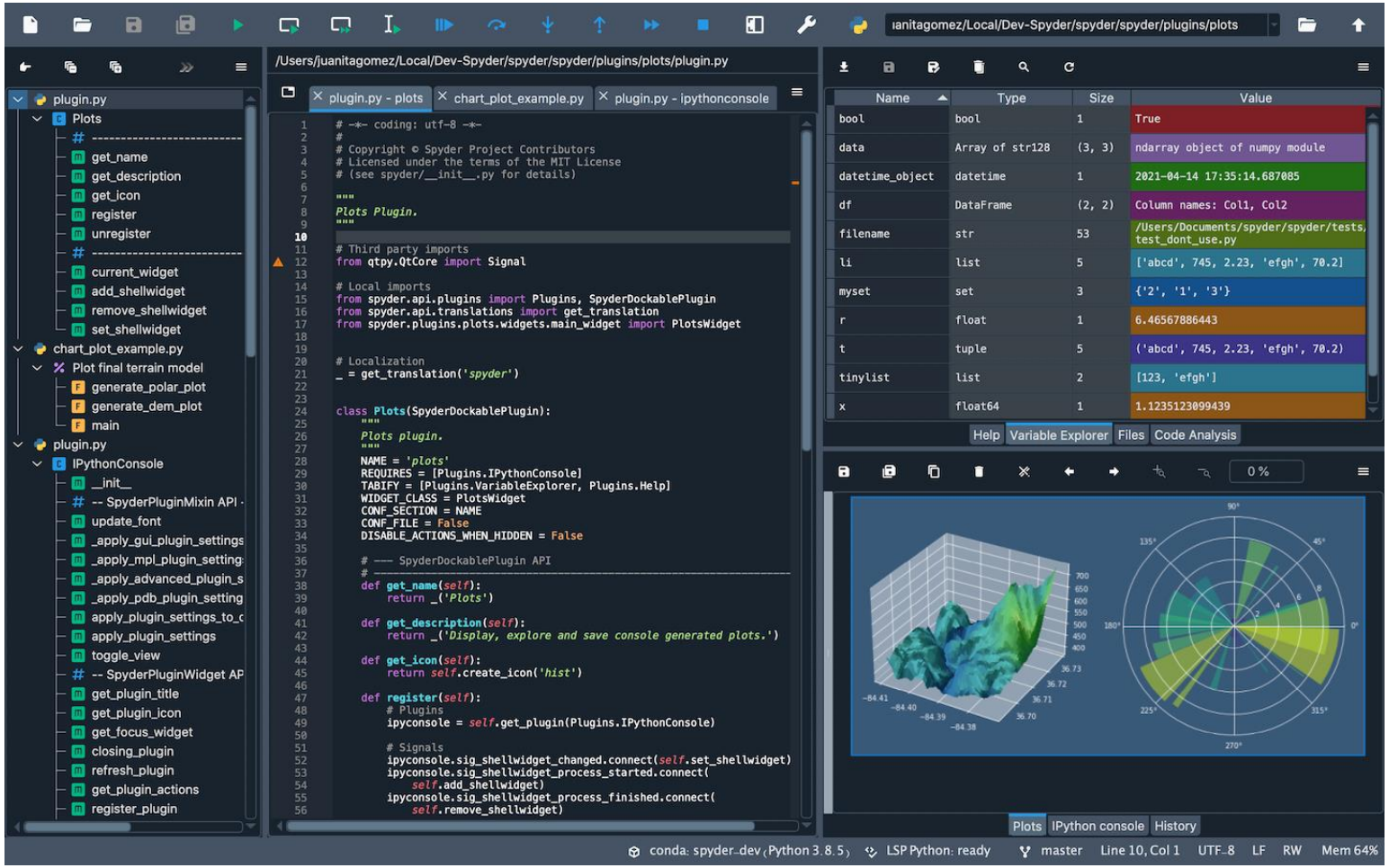
Một vài IDE cho ngôn ngữ Python

- PyCharm là IDE đa nền tảng được phát triển bởi JetBrains.
- Một vài tính năng của PyCharm:
  - Tính năng hoàn thành mã thông minh
  - Kiểm tra mã, đánh dấu lỗi và sửa lỗi nhanh chóng
  - Có màu sắc cho từng loại keyword.
- Pycharm hỗ trợ các framework như Django, Flask, Google App Engine, Pyramid và web2py.
- PyCharm hỗ trợ ngôn ngữ lập trình JavaScript, CoffeeScript, TypeScript, SQL, HTML / CSS…

Open Source Technologies

Một vài IDE cho ngôn ngữ Python

spyder



Một vài IDE cho ngôn ngữ Python

- Phần mềm đa nền tảng: windows, Linux, Mac
- Miễn phí
- Spyder được thiết kế để tích hợp với các phần mềm mã nguồn mở như Matplotlib, SciPy, NumPy, Pandas, SymPy…
- Spyder có sự kết hợp giữa chức năng chỉnh sửa, phân tích, gỡ lỗi
- Hỗ trợ khả năng khám phá dữ liệu, thực thi tương tác, kiểm tra và trực quan hóa dữ liệu.

Một vài IDE cho ngôn ngữ Python

Visual Studio Code

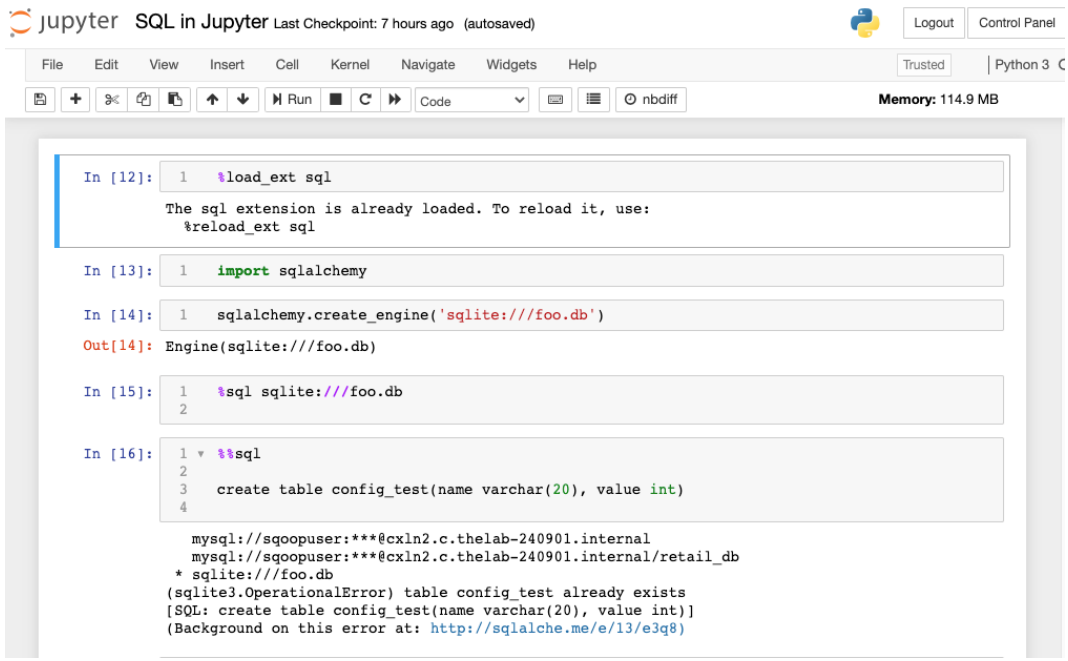
- Mặc định Visual Studio Code không hỗ trợ trình thông dịch Python.
- Cái đặt them package Python.

Open Source Technologies

Một vài IDE cho ngôn ngữ Python

jupyter

- Môi trường phát triển dựa trên giao diện web.
- Hỗ trợ markdown, notebook và code trên cùng 1 file.
- Cho phép user cầu hình và sử dụng linh hoạt trong data science, science computing và Machine Learning.
- Cài đặt Jupyter Notebook
  - *pip install notebook*
  - *conda install -c conda-forge notebook*



2 Cơ bản về ngôn ngữ Python

Open Source Technologies

colab.research.google.com/?utm_source=scs-index

Welcome To Colaboratory

File Edit View Insert Runtime Tools Help

Share Sign in

Table of contents

Getting started
Data science
Machine learning
More Resources
Machine Learning Examples

Section

+ Code + Text Copy to Drive Connect Editing

What is Colaboratory?

Colaboratory, or "Colab" for short, allows you to write and execute Python in your browser, with

- Zero configuration required
- Free access to GPUs
- Easy sharing

Whether you're a **student**, a **data scientist** or an **AI researcher**, Colab can make your work easier. Watch Introduction to Colab to learn more, or just get started below!

Getting started

The document you are reading is not a static web page, but an interactive environment called a **Colab notebook** that lets you write and execute code.

For example, here is a **code cell** with a short Python script that computes a value, stores it in a variable, and prints the result:

```
[ ] seconds_in_a_day = 24 * 60 * 60
    seconds_in_a_day
```

Cài đặt Python trên Linux

Mở terminal, gõ lệnh sau:

sudo apt update

sudo apt install python

Kiểm tra version của Python

python -V

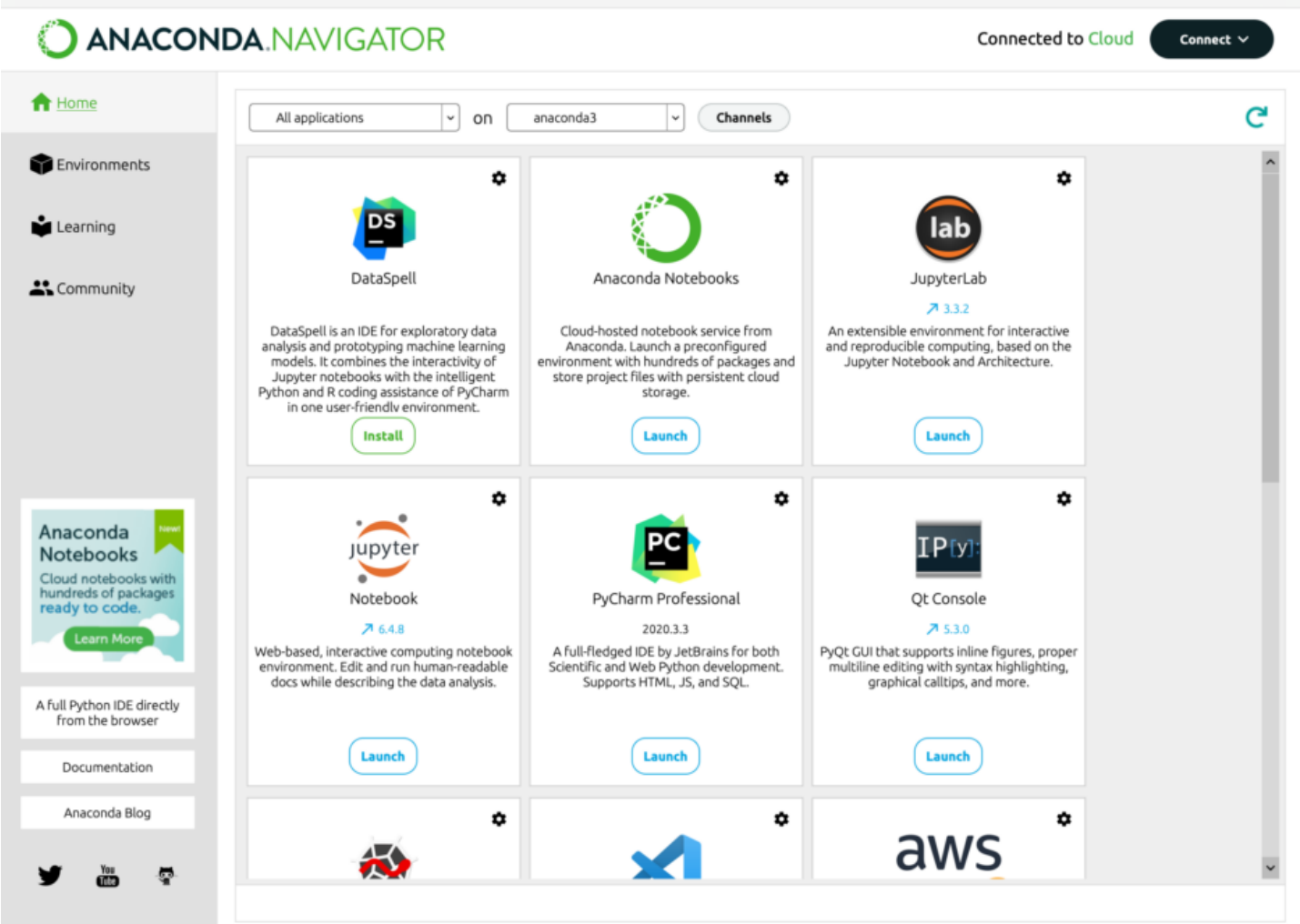
Quản lý các package

- pip viết tắt của chữ Package Instasler for Python.
- Là công cụ quản lý các package Python.
- pip được cài đặt mặc định khi cài Python.
- pip –version => Xem phiên bản của pip
- pip install tên_package
- pip uninstall tên_package
- pip list => Xem các package

Open Source Technologies

Quản lý các package

ANACONDA



2 Cơ bản về ngôn ngữ Python

Một vài điều cơ bản để bắt đầu

- input(): Nhận dữ liệu nhập vào từ bàn phím
- print(): Xuất nội dung ra màn hình
- \n: xuống dòng
- \t: cách nhau một dấu tab
- #: comment trong ngôn ngữ Python
- Chạy file python từ terminal: python tên_file.py

Ví dụ cơ bản

Nhập vào họ tên và xuất ra xin chào, sử dụng cách nối 2 biến

```
ho = input("Họ của bạn là gì? ")
ten = input("Tên của bạn là gì? ")

print("Xin chao ", ho, ten)

print("Xin chao ", ho + " " + ten)
```

Ví dụ cơ bản

Nhập vào 2 số, tính tổng của hai số

```
# Nhập hai số từ bàn phím
so1 = input("Nhập số thứ nhất: ")
so2 = input("Nhập số thứ hai: ")

# Thực hiện phép cộng
tong = int(so1) + int(so2)

# Hiển thị kết quả
print("Tổng của", so1, "và", so2, "là:", tong)
```

3 Kiểu dữ liệu

Biến trong Python

- Quy tắc đặt tên biến trong Python
  - Bắt đầu bằng chữ hoặc ký tự gạch dưới.
  - Không bắt đầu bằng số
  - Không có khoảng trắng hoặc ký tự đặc biệt.
  - Chỉ bao gồm ký tự chữ, số, dấu gạch dưới
  - Phân biệt chữ hoa chữ thường
- Không cần khai báo kiểu dữ liệu khi khai báo biến
- Sử dụng dấu = để gán giá trị cho biến.
- Để xem kiểu dữ liệu của biến: type(tên_biến)

3 Kiểu dữ liệu

Biến trong Python

Mộ vài ví dụ đặt tên và khởi tạo biến:

```
Mon_hoc = “Cong Nghe MMM”
mon_hoc = “Mã Nguồn Mở”
a = 10
b = 15.5
x = y = z = 100    => gán giá trị cho nhiều biến
x,y,z = 10,20,15
```

In giá trị và kiểu dữ liệu của biến

```
print(Mon_hoc)
print(type(a))
print(type(b))
print(x, y, z)
print(a + b)
```

Biến trong Python

Phạm vi của biến:

- Biến local
- Biến global: Sử dụng từ khóa global để khai báo biến global.

Ví dụ phạm vi của biến

```
def giatri():
        global a
        a = 10

giatri()
print("Gia tri cua a la: ", a)
```

3 Kiểu dữ liệu

Các kiểu dữ liệu trong Python

- Numbers: Integer, Complex number, Float
- Sequence Type: String, list, tuple
- Boolean
- Set
- Dictionary

3 Kiểu dữ liệu

Một vài ví dụ về các kiểu dữ liệu

x = “Môn công nghệ MMM"	string
x = 20	int
x = 20.5	float
x = 1j	complex
x = ["apple", "banana", "cherry"]	list
x = ("apple", "banana", "cherry")	tuple
x = range(6)	range
x = {"name" : "John", "age" : 36}	dictionary
x = {"apple", "banana", "cherry"}	set

3 Kiểu dữ liệu

Kiểu dữ liệu Number

Khai báo

- a = 5 => kiểu int
- b = 5.6 => kiểu float
- x = -10 => kiểu int
- y = 10E4 => kiểu float

Ép kiểu

- a = 10.2
- b = 20
- c = “45”
- int(a) => kết quả = 10
- float(b) => kết quả = 20.0
- int(c) => kết quả = 45

Kiểu dữ liệu Sequence

Kiểu String

- Sử dụng dấu “” hoặc ‘’ để khởi tạo chuỗi.
- Trong PyThon, string được xem là mảng.
- Có thể sử dụng dấu [ ] để truy xuất các phần tử trong chuỗi.
- Thứ tự trong chuỗi:

0	1	2	3	4	5	6	7	8
M	Ã	N	G	U	Ồ	N	M	Ở
-9	-8	-7	-6	-5	-4	-3	-2	-1

Ví dụ: chuoi = “Mã nguồn mở”

print(chuoi[1]) => Kết quả Ã

Kiểu dữ liệu Sequence

Phương thức đối với kiểu String

- Lấy chiều dài chuỗi: len()

  Ví dụ: len("Hello") => kết quả = 5

- Kiểm tra chuỗi con có tồn tại hoặc không tồn tại trong chuỗi. Kết quả trả về là True hoặc False

  Ví dụ: str = "Công nghệ mã nguồn mở"

  "mã" in str => kết quả trả về = true

  "mở" not in str => kết quả trả về = false

Kiểu dữ liệu Sequence

Phương thức đối với kiểu String

- Lấy chuỗi con

  Ví dụ: str = “Công nghệ mã nguồn mở”

  print(str[5:8] => kết quả = “nghệ”

  print(str[:4]) => kết quả = “Công”

  print(str[5:]) => kết quả = “nghệ mã nguồn mở”

- upcase(): chuyển thành chữ in hoa. Ví dụ: str.upcase()
- lowercase(): chuyển thành chữ in thường. Ví dụ: str.lowercase()
- strip(): bỏ khoảng trắng ở đầu và cuối choỗi. Ví dụ: a.strip()

Kiểu dữ liệu Sequence

Phương thức đối với kiểu String

- Thay thế chuỗi
- Phân chia chuỗi

Ví dụ: str = “Công nghệ mã nguồn mở”

str.replace(“Công”,”công”)

str.split(“ ”)

- Nối chuỗi: sử dụng dấu +

Kiểu dữ liệu Sequence

Kiểu List

- List có thể chứa các giá trị khác kiểu dữ liệu.
- List có thể thay đổi giá trị.
- Giá trị trong list có thể trùng nhau
- Các giá trị trong list có thể được sắp xếp
- Sử dụng dấu [ ] để khai báo list

Ví dụ: monhoc = [“CSDL”, “MMT”, “NMTH”]

danhsach = [“QTM”, 5, 20.5, “CTDL”]

Kiểu dữ liệu Sequence

Phương thức trên List

- Lấy chiều dài của list
- Lấy giá trị các phần tử trong list.
- Thay đổi giá trị trong list.
- Thêm giá trị vào list
- Nối 2 list thành 1 list
- Xóa phần tử trong list
- Sắp xếp, copy, đảo ngược list

Kiểu dữ liệu Sequence

Kiểu dữ liệu Tuple

- Dùng để lưu một dãy các giá trị
- Có thể lưu các kiểu dữ liệu khác nhau trong cùng một tuple
- Giá trị trong tuple có thể sắp xếp
- Giá trị trong tuple không thể thay đổi.
- Trong tuple, giá trị có thể trùng nhau.
- Vị trí trong tuple bắt đầu bằng 0.

Ví dụ: tup1 = (“xanh”, “vàng ”, “đỏ”)

Kiểu dữ liệu Sequence

Phương thức trên Tuple

- Truy xuất giá trị
- Kiểm tra 1 giá trị có tồn tại trong list
- Nối các tuple
- Đếm phần tử xuất hiện bao nhiêu lần trong tuple

Kiểu dữ liệu Set

- Kiểu dữ liệu tập hợp, dùng để lưu 1 dãy các giá trị
- Các giá trị có thể khác kiểu dữ liệu.
- Các giá trị trong set không được sắp xếp, không được thay đổi
- Các giá trị trong set có thể xóa, thêm mới.
- Trong set, các giá trị không được trùng nhau.
- Dùng dấu { } để tạo set

Ví dụ: set1 = {“Toan”, “AnhVan”, 5, 20}

Phương thức trên Set

- Truy xuất giá trị
- Thêm phần tử vào set
- Thêm một set vào một set khác
- Xóa phần tử trong set

Kiểu dữ liệu Dictionary

- Kiểu dữ liệu dictionary dung để lưu dữ liệu có dạng key:value
- Các giá trị có thể được sắp xếp.
- Các giá trị có thể thay đổi, update. Không cho phép trùng.
- Ví dụ kiểu dữ liệu dictionary

```
sinhvien{
    “HoTen” : “Nguyen Van Tho”,
    “Quequan” : “HCM”,
    “namsinh” : 2000
}
```

Phương thức trên Dictionary

- Truy xuất giá trị
- Lấy tên của các key trong dictionary
- Lấy tất cả giá trị trong dictionary
- Thay đổi giá trị
- Thêm cặp giá trị key:value
- Xóa cặp giá trị, xóa tất cả
- Kiểm tra key đã có trong dictionary hay không

Phương thức trên Dictionary

- Nested Dictionaries là gì?
- Cho ví dụ về nested dictionaries
- Thay đổi, cập nhật giá trị trong nested dictionaries

4 Toán tử và biểu thức

Toán tử trong ngôn ngữ Python

- Toán tử số học
- Toán tử gán
- Toán tử so sánh
- Toán tử logic
- Toán tử định danh
- Toán tử thành viên

4 Toán tử và biểu thức

Toán tử số học

Toán tử	Phép toán	Ví dụ
+	Cộng	x + y
-	Trừ	x - y
*	Nhân	x * y
/	Chia	9 / 2 = 4.5
%	Chia lấy dư	x % y
**	Lũy thừa	x ** y
//	Chia lấy nguyên	9 // 2 = 4

Toán tử gán

Toán tử	Ví dụ	Tương đương
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3
//=	x //= 3	x = x // 3
**=	x **= 3	x = x ** 3

Toán tử so sánh

Toán tử	Phép toán	Ví dụ
==	So sánh bằng	x == y
!=	So sánh không bằng	x != y
>	So sánh lớn hơn	x > y
<	So sánh nhỏ hơn	x < y
>=	Lớn hơn hoặc bằng	x >= y
<=	Nhỏ hơn hoặc bằng	x <= y

Toán tử logic

Toán tử	Mô tả	Ví dụ
and	Đúng khi cả hai điều kiện đều đúng	x < 5 and x < 10
or	Sai khi cả hai điều kiện đều sai	x < 5 or x < 4
not	Lấy phủ định	not(x < 5 and x < 10)

Toán tử định danh

Toán tử	Mô tả	Ví dụ
is	Trả về true nếu 2 biến cùng đối tượng	x = 5 y = 5.0 x is y => kết quả false
is not	Trả về true nếu 2 biến không cùng đối tượng	

Toán tử member

Toán tử	Mô tả	Ví dụ
in	Trả về true nếu giá trị tồn tại	a = (2, 5, 7, 10, 9, 15) 2 in a => kết quả true
not in	Trả về true nếu giá trị không tồn tại	b = "Good morning!“ "good“ not in b => kết quả true

5 Cấu trúc điều khiển

Cấu trúc điều kiện if

```
so1 = 60
so2 = 50
if so1 > so2:
    print("So1 lớn hơn so2")
```

```
so1 = 50
so2 = 60
if so1 > so2:
    print("So1 lớn hơn so2")
else:
    print("So1 không lớn hơn so2")
```

5 Cấu trúc điều khiển

Cấu trúc điều kiện if

```
so1 = 60
so2 = 60
if so1 > so2:
    print("So1 lớn hơn so2")
elif so1 == so2:
    print("So1 bằng so2")
```

```
so1 = 30
so2 = 60
if so1 > so2:
    print("So1 lớn hơn so2")
elif so1 == so2:
    print("So1 bằng so2")
else:
    print("So1 nhỏ hơn so2")
```

5 Cấu trúc điều khiển

Cấu trúc điều kiện if…else lồng

```
x = 50
if x > 20:
    print("X = ",x,"lớn hơn 20")
    if x > 60:
        print("X lớn hơn 60")
    else:
        print("X nhỏ hơn 60")
```

Câu lệnh sau sẽ gây ra lỗi Indentation

```
x = 50
if x > 20:
print("X = ",x,"lớn hơn 20")
```

Cấu trúc điều kiện if…else lồng

```
chuoi = "Khoa công nghệ thông tin"
word = "công"
if word not in chuoi:
    print("Chữ",word, "không tồn  trong chuỗi ",chuoi)
else:
    print("Chữ ",word,"có trong chuỗi ",chuoi)
```

5 Cấu trúc điều khiển

Cách viết ngắn gọn của câu điều kiện if

```
so1=5

so1=6

print("so1 bằng so2") if a==b else print("so1 không bằng so2")


print("so1 bằng so2") if a==b else print("so1 nho hơn so2") if
a<b else print("so1 lớn hơn so2")
```

5 Cấu trúc điều khiển

Cấu trúc lặp while

```
i = 1
while i <= 10:
    print("giá trị:", i)
    i += 1
```

```
# Xuất ra các giá trị chẵn trong
khoản từ 1 – 20

i = 1
while i <= 20:
    if i%2 == 0:
        print(i)
    i += 1
```

5 Cấu trúc điều khiển

Cấu trúc lặp while

```
i = 1
while i < 10:
  print(“Giá trị ”, i)
  if i == 3:
    break
  i += 1
```

```
i = 0
while i < 10:
  i += 1
  if i == 5:
    continue
  print(“Giá trị ”, i)
```

Lệnh break sẽ thoát vòng lặp while
Lệnh coutinue thực hiện lệnh tiếp theo

5 Cấu trúc điều khiển

Open Source Technologies

Cấu trúc lặp while

```
string = "Đại Học Mở thành phố Hồ Chí Minh"
i = 0
while i < len(string.split()):
    print(string.split()[i])
    i += 1
    if string.split()[i] == 'thành':
        break
```

5 Cấu trúc điều khiển

Cấu trúc lặp for

```
danhsach = [4, 7, 8, 3, 10, 23, 11]
for so in danhsach:
    print(so)
```

```
chuoi = "Khoa công nghệ thông tin"
for word in chuoi.split():
    print(word)
```

Cấu trúc lặp for

Hàm range() trả về một dãy số có thứ tự

```
for x in range(10):
  print(x)
```

=> Kết quả trả về từ 0 – 9

```
for x in range(2, 10):
  print(x)
```

=> Kết quả trả về từ 2 – 9

```
for x in range(2, 20, 3):
  print(x)
```

=> Kết quả: 2 5 8 11 14 17

Open Source Technologies

Cấu trúc lặp for

Hai cách viết khác nhau khi áp dụng hàm range()

```
danhsach = [4,7,8,3,10,23,11]
sum = 0
for so in danhsach:
    print(so)
    sum += so
print("Tổng:", sum)
```

```
danhsach = [4,7,8,3,10,23,11]
sum = 0
for so in range(len(danhsach)):
    print(danhsach[so])
    sum += danhsach[so]
print("Tổng: ", sum)
```

Vòng lặp for lồng nhau

```
khoi = ["1", "2", "3"]
lop = ["1", "2", "3", "4", "5"]
for x in khoi:
    for y in lop:
        tenlop = x+y
        print("Lớp:",tenlop)
```

Cấu trúc switch…case

- Trong Python, không có cấu trúc switch…case ở các version 3.10 trở về trước.
- Sử dụng dictionary hoặc câu lệnh if để giải quyết.
- Switch…case chỉ có từ version 3.10 trở về sau.

```
match so:
        case 0:
            return "Số 0"
        case 1:
            return "Số một"
        case 2:
            return "Số hai"
        case default:
            return "Không phải số"
```

Cấu trúc switch…case

```
def switch(lang):
    if lang == "JavaScript":
        return "Ngôn ngữ JavaScript"
    elif lang == "PHP":
        return "Ngôn ngữ PHP"
    elif lang == "Python":
        return "Ngôn ngữ Python"
    else:
        return "Giá trị không hợp lệ"

ngonngu = input("Nhập vào ngôn ngữ")
print(switch(ngonngu))
```

Cấu trúc switch…case

```
lang = input("Nhập vào ngôn ngữ")

match lang:
    case "JavaScript":
        print("Ngôn ngữ JavaScript.")
    case "Python":
        print("Ngôn ngữ Python")
    case "PHP":
        print("Ngôn ngữ PHP")
    case _:
        print("Giá trị nhập vào không hợp lệ")
```

6 Định nghĩa hàm

Hàm trong Python

- Dùng hàm để gom các khối code thành một đoạn riêng biệt.
- Có thể tái sử dụng một cách dễ dàng.
- Có 2 loại hàm: Built-in library, User-defined
- Kết quả của một hàm có thể được gán cho một biến.
- Dùng từ khóa def để khai báo hàm.

```
def ten_ham():
    các lệnh trong hàm
```

6 Định nghĩa hàm

Hàm trong Python

```
def hello():
    print("Xin chào")
# Gọi hàm
hello()
# Sử dụng hàm main
if __name__ == "__main__":
    print("Xuất ra nội dung trong hàm hello")
    hello()
```

6 Định nghĩa hàm

Hàm trong Python

```
def hello(a):
  print("Xin chào",a)

#Truyền đối số trong hàm
hello("Python")
```

```
def tong(a,b):
    return a+b

#Gán kết quả cho biến
tong = tong(5,4)
print(tong)
```

Đối số mặc định là gì?

6 Định nghĩa hàm

- Phân biệt phương thức và hàm?
- Biến cục bộ và biến toàn cục.

6 Định nghĩa hàm

Hàm lambda()

- Lambda là một hàm ẩn hay hàm anonymous trong Python.
- Được sử dụng khi cần thực hiện một công việc đơn giản
- Hàm lambda chỉ chứa một biểu thức
- Được viết dưới 1 dòng duy nhất.
- Hàm lambda không có khả năng tái sử dụng.
- Cú pháp: *lambda arguments : expression*

```
x = lambda a : a + 10
print(x(5))
```

6 Định nghĩa hàm

Hàm lambda()

Lambda kết hợp với filter

```
list1 = [7, 1, 2, 8, 17, 3, 10, 14]
list2 = list(filter(lambda x: (x%2 != 0), list1))
```

Lambda kết hợp với map

```
list1 = [3, 4, 5, 9, 8, 7, 11, 13]
list2 = list(map(lambda x: x * 2, list1))
```

7 Xử lý file

Xử lý file text

- Sử dụng hàm open() để mở file text
- Một số mode trong hàm open()
  - r: Đọc nội dung file
  - a: Thêm nội dung vào file
  - w: Mở file để thêm nội dung mới vào file
  - x: Tạo file mới
- Một số hàm thường sử dụng trong xử ý file text: read(), readline(), readlines(), write(), close()

7 Xử lý file

Xử lý file text

Ví dụ:
```
c = open(“file.txt”, “r”)
print(c.read())
```

=> Phải close file sau khi làm việc

Ví dụ:
```
c = open(“file.txt”, “r”, encoding='utf-8')
print(c.read())
```

Ví dụ:
```
with open("file.txt") as f:
data = f.read()
```

=> Không cần phải close file sau khi làm việc

7 Xử lý file

Xử lý file text

Thêm nội dung vào file

```
file = open(‘file.txt','w')
file.write(“Công cha như núi Thái Sơn \n")
file.write(“Nghĩa mẹ như nước trong nguồn chảy ra")
file.close()
```

Lưu ý: Với mode “w”, nội dung mới sẽ lưu đè lên nội dung cũ.

7 Xử lý file

Xử lý file text

Thêm nội dung mới vào file

```
file = open(‘file.txt',’a')
file.write(“Cho tròn chữ hiếu mới là đạo con.")
file.close()
```

Lưu ý: append sẽ them nội dung vào cuối file

7 Xử lý file

Xử lý file .csv

Cách 1:

```
with open('university_records.csv', ‘r’) as f:
 print(f.read())
```

Cách 2:

```
f = open("university_records.csv", ‘r’)
print(f.read())
```

7 Xử lý file

Xử lý file .csv

Ký tự	Mode	Mô tả
‘r’	Read (Mặc định)	Mở file để đọc
‘w’	Write	Mở file để ghi
‘a’	Append	Mở file để thêm dữ liệu
‘r+’	Read+Write	Mở file vừa đọc + ghi
‘x’	Create	Tạo file mới